

BTC ORDER FLOW LITE

Comprehensive Product Architecture & Build Document

Version 2.0 | Multi-Exchange Real-Time Order Flow System

Document Type	Product Architecture & Engineering Blueprint
Status	Active — Ready to Build
Target Exchanges	Binance (Primary) · OKX (Secondary) · Bybit (Secondary)
MVP Scope	Phase 1: Binance live footprint table (terminal)
Tech Stack	Python · asyncio · WebSocket · React (frontend)
Author Role	Lead Product Architect & Systems Designer
Revision	v2.0 — Expanded from v1.0 concept brief

1. Executive Summary & Core Mission

BTC Order Flow Lite is a lightweight, real-time order flow visualization system designed to expose actual market microstructure to retail traders. It aggregates executed trade data from the three highest-volume BTC perpetual futures exchanges — Binance, OKX, and Bybit — and renders it into a readable footprint format that reveals who is actually in control at each price level.

This is not a signal generator. It is a market transparency layer — a system that forces traders to engage with real price discovery mechanics rather than lagging derived indicators.

1.1 The Core Problem (Precise Diagnosis)

Problem Layer	Manifestation in Markets
Indicator Dependency	Traders react to RSI/MACD signals that are derivatives of price, not causes. They are always late.
Lack of Context	No visibility into whether a move is driven by real volume or thin air. Entries lack confirmation.
Tool Inaccessibility	Professional footprint tools (Bookmap, Sierra Chart) cost \$100–\$300/month and have steep learning curves.
Signal Provider Reliance	Retail traders outsource thinking, creating dependency and inability to read price independently.
Exchange Blindspot	Most free tools show one exchange only. BTC price is a multi-exchange consensus — single-source is incomplete.

1.2 Mission Statement

 PRODUCT MISSION
To democratize order flow analysis for retail BTC traders by building a fast, affordable, and educational tool that exposes real executed market activity across Binance, OKX, and Bybit — teaching traders to read markets, not follow signals.

2. Exchange Architecture & Data Reality

2.1 Why These Three Exchanges

Binance, OKX, and Bybit collectively represent the dominant share of BTC perpetual futures volume globally. Including all three is not optional for accuracy — it is the minimum viable multi-exchange setup. Here is the reasoning:

Exchange	Role	Est. BTC Perp Volume	Key Characteristics
Binance	Primary	\$10–20B/day	Deepest liquidity, most retail + institutional flow, USDT-margined BTC perp (BTCUSDT)
OKX	Secondary	\$4–8B/day	Strong Asian session dominance, sophisticated traders, significant funding rate divergence events
Bybit	Secondary	\$3–6B/day	Fast growing, popular with retail, tends to lead short-term moves due to overleveraged positions

2.2 Critical Data Realities (Architect's Warnings)

 KNOWN LIMITATIONS — BUILD WITH THESE IN MIND, NOT AROUND THEM
1. Side Classification: Exchanges mark buys/sells inconsistently. Binance uses aggressor-side tagging. OKX and Bybit may differ. Normalize carefully — do not assume parity. 2. Timestamp Skew: WebSocket delivery latency varies per exchange. Merging trades by timestamp will produce small mismatches (50–300ms). This is acceptable — directional clarity matters, not microsecond precision. 3. Trade Aggregation Differences: Binance aggregates multiple fills into one trade event (aggTrade). OKX sends raw fills. This means Binance trade counts look smaller. Normalize by volume, not count. 4. Funding Rate Divergence: Different funding rates between exchanges can drive arbitrage flows that appear as genuine directional pressure. Flag high divergence events as noise context. 5. Exchange Downtime: Any exchange can have WebSocket drops. Implement reconnect logic with exponential backoff from day one.

2.3 Trade Data Schema (Normalized)

All exchanges must be normalized into this canonical schema before any processing occurs:

Field	Type	Source	Notes
exchange	str	Injected	"binance" "okx" "bybit"
symbol	str	Stream	Normalized to "BTC-PERP-USDT"
price	float	Raw	Execution price, no rounding
volume	float	Raw	In BTC (base asset), not USD

Field	Type	Source	Notes
side	str	Raw / Inferred	"buy" "sell" — aggressor side
timestamp	int	Raw	Unix ms — exchange time preferred
trade_id	str	Raw	For deduplication (Binance aggTrade)
raw	dict	Raw	Original payload preserved for debugging

3. System Architecture (Detailed)

The system is structured as a unidirectional data pipeline: raw exchange streams flow through ingestion, normalization, aggregation, detection, and finally render. Each layer has a single responsibility and can be tested independently.

3.1 Pipeline Overview

Layer	Component	Responsibility	Technology
L1	WebSocket Clients	Connect to exchange streams, handle reconnects, emit raw events	Python asyncio + websockets
L2	Normalizer	Map raw exchange payload → canonical Trade schema	Python dataclasses
L3	Trade Bus	In-memory queue routing normalized trades to consumers	asyncio.Queue
L4	Aggregation Engine	Group trades into price buckets, compute buy/sell/delta/imbalance	Python dict / deque
L5	Detection Engine	Run pattern detection (imbalance, absorption, exhaustion)	Pure Python logic
L6	State Manager	Maintain rolling time-windowed footprint state	Python + circular buffer
L7	Output Adapter	Serialize state to terminal / WebSocket / REST for frontend	Rich (terminal) / FastAPI

3.2 WebSocket Connection Architecture

Each exchange runs as an independent async task. A supervisor manages reconnects:

Exchange	Stream Endpoint	Format	Reconnect Strategy
Binance	wss://fstream.binance.com/ws/btcusdt@aggTrade	aggTrade JSON	Exponential backoff, max 60s
OKX	wss://ws.okx.com:8443/ws/v5/public → trades.BTC-USDT-SWAP	push array	Exponential backoff + ping/pong
Bybit	wss://stream.bybit.com/v5/public/linear → publicTrade.BTCUSDT	delta array	Exponential backoff + heartbeat

3.3 Aggregation Engine Design

The aggregation engine is the mathematical core of the product. It must be correct before anything else is built on top of it.

Concept	Definition & Implementation
Price Bucket	A configurable price range (default: \$1 per bucket for BTC perps). Trades are binned by $\text{floor}(\text{price} / \text{bucket_size}) * \text{bucket_size}$.
Buy Volume	Sum of volume where side == "buy" within the time window for this bucket.
Sell Volume	Sum of volume where side == "sell" within the time window for this bucket.
Total Volume	Buy Volume + Sell Volume. Used for absorption detection threshold.
Delta	Buy Volume – Sell Volume. Positive = buy pressure. Negative = sell pressure.
Imbalance %	$ \text{Delta} / \text{Total Volume} \times 100$. Expressed directionally: +75% = buy dominant.
Time Window	Configurable rolling window (1, 5, 15 min). Implemented as a deque of (timestamp, trade) pairs. Trades older than window_seconds are evicted on each cycle.
Multi-Exchange Merge	Aggregation runs on the combined normalized trade bus. All three exchanges contribute to the same bucket. Source exchange is retained for optional per-exchange breakdown.

4. Detection Engine — Pattern Logic

Detection is applied after aggregation. These are not signals — they are contextual annotations that help traders understand what the footprint data means. Each detection must have a clear educational explanation attached.

4.1 Imbalance Detection

Parameter	Detail
Trigger Condition	Imbalance % $\geq$ configurable threshold (default: 70%)
Buy Dominant	Delta > 0 AND imbalance $\geq$ threshold $\rightarrow$ highlight green
Sell Dominant	Delta < 0 AND imbalance $\geq$ threshold $\rightarrow$ highlight red
Minimum Volume Filter	Total volume at bucket must exceed floor (e.g., 0.5 BTC) to filter noise on thin levels
Educational Label	"Aggressive buyers/sellers dominating this level — passive orders being consumed"
Actionable Context	High imbalance near a key support/resistance level signals conviction. In isolation it is noise.

4.2 Absorption Detection

Parameter	Detail
Definition	Large volume transacted with minimal resulting price movement — implies large passive orders absorbing aggression
Trigger Condition	Total volume at bucket $>$ absorption_volume_threshold AND price range over window $<$ absorption_price_threshold
Default Thresholds	Volume: top 20th percentile of all buckets in window. Price movement: $< 0.05\%$ of BTC price
Implementation	Track high-water and low-water price within bucket during window. If range $<$ threshold AND volume qualifies, flag.
Visual Treatment	Bold border or distinct background on flagged bucket. Do not use color (conflicts with buy/sell coloring).
Educational Label	"High volume, low movement — a large player may be defending this price level"

Parameter	Detail
False Positive Risk	Can trigger in low-volatility periods with moderate volume. Requires context — cross-reference with overall market volatility.

4.3 Exhaustion Detection

Parameter	Detail
Definition	A volume spike in one direction followed by a reversal of that directional pressure within the same window
Trigger Condition	Step 1: identify a bucket with peak buy or sell volume (top 10th percentile). Step 2: in the subsequent 30–60 seconds, check if the opposing side volume at that price increases significantly (>40% of the original spike).
Implementation Note	Requires time-series awareness. Store timestamped volume deltas per bucket to detect the reversal pattern.
Educational Label	"Volume spike followed by counter-pressure — momentum may be weakening at this level"
Caution	This is the most complex detection to implement correctly. Build imbalance detection first. Add exhaustion in Phase 3 only after footprint accuracy is verified.

5. Core Features (MVP → Full Product)

5.1 Feature Matrix

Feature	Phase	Priority	Description
Binance Live Trade Stream	Phase 1	P0 — Critical	WebSocket ingestion + normalization
Price Bucket Aggregation	Phase 1	P0 — Critical	Footprint engine, buy/sell/delta/imbalance
Terminal Footprint Display	Phase 1	P0 — Critical	Rich-formatted live table in terminal
Time Window Control (1/5/15m)	Phase 1	P0 — Critical	Rolling window with configurable granularity

Feature	Phase	Priority	Description
OKX + Bybit Integration	Phase 2	P1 — High	Multi-exchange merge into unified view
Per-Exchange Breakdown	Phase 2	P2 — Medium	Optional: see each exchange contribution
Imbalance Highlighting	Phase 3	P0 — Critical	Color-coded dominance visualization
Absorption Detection	Phase 3	P1 — High	High volume / low movement flagging
Exhaustion Detection	Phase 3	P2 — Medium	Reversal pressure identification
Educational Tooltips/Labels	Phase 3	P1 — High	Inline explanations per detected pattern
Web Dashboard (React)	Phase 4	P1 — High	Browser-based footprint visualization
Vertical Heatmap View	Phase 4	P2 — Medium	Alternative to table view
Alert System	Phase 4	P3 — Low	Configurable threshold alerts (no signals)
Historical Replay	Phase 5	P3 — Low	Replay stored trade data for backtesting reads

5.2 Footprint Table Specification

The footprint table is the primary output surface. Every column must earn its place:

Column	Content	Width	Notes
Price	Bucket price level	Fixed	Sorted descending (highest price = top)
Buy Vol	Cumulative buy volume (BTC)	Fixed	Green text. Show in BTC, not USD.
Sell Vol	Cumulative sell volume (BTC)	Fixed	Red text.
Delta	Buy – Sell	Fixed	Signed: +120.4 or –88.2. Color by sign.
Imbalance	Directional imbalance %	Fixed	+75% (green) or –78% (red). Blank if < min volume.

Column	Content	Width	Notes
Flags	Detection annotations	Fixed	Symbols: [ABS] absorption, [EXH] exhaustion
Total Vol	Buy + Sell volume	Fixed	Secondary column — useful for context

EXAMPLE FOOTPRINT OUTPUT (MVP Terminal Display)					
Price	Buy Vol	Sell Vol	Delta	Imbalance	Flags
67,250	42.80	12.10	+30.70	+78% ▲	
67,249	18.40	62.30	-43.90	-77% ▼	[ABS]
67,248	8.20	9.10	-0.90	--	
67,247	91.00	22.50	+68.50	+80% ▲	

6. Technical Specification

6.1 Technology Stack — Justified

Component	Technology	Justification	Alternatives Considered
Async Runtime	Python asyncio	Native async support, excellent WS library ecosystem, fast iteration speed	Rust tokio (overkill for MVP), Node.js (worse data libs)
WebSocket Client	websockets library	Battle-tested, supports ping/pong, works with asyncio natively	aiohttp, websocket-client (sync only)
Data Processing	Python dicts + deque	Zero dependencies for MVP. Deque for O(1) window eviction. Optimize later.	Pandas (too heavy for real-time), numpy (added later)
Terminal UI	Rich (Python)	Beautiful formatted tables, live updates, color support, zero front-end work	Curses (complex), Print (unusable at speed)
Web Backend	FastAPI	Async-native, WebSocket support, automatic OpenAPI docs, fast to build	Flask (sync), Django (heavy)
Web Frontend	React + Vite	Component model fits footprint table perfectly. Vite for fast dev loop.	Streamlit (too limited for custom footprint UI)

Component	Technology	Justification	Alternatives Considered
Configuration	TOML / .env	Human-readable config for thresholds, bucket sizes, exchange toggles	YAML (fine), JSON (no comments)
Logging	Python logging + structlog	Structured logs essential for debugging multi-exchange timing issues	Print statements (unacceptable in production)

6.2 Project Structure

Repository layout from day one. Discipline in structure prevents technical debt:

Path	Contents
btc-orderflow/	Project root
├── main.py	Entry point — starts all async tasks
├── config.toml	User-configurable: bucket size, thresholds, time windows, exchanges
├── ingestion/	WebSocket clients: binance.py, okx.py, bybit.py
└── base.py	Abstract base class for all exchange clients
├── normalization/	Trade schema: models.py, normalizer per exchange
├── aggregation/	Bucket engine: engine.py, time_window.py
├── detection/	Pattern detectors: imbalance.py, absorption.py, exhaustion.py
├── state/	FootprintState manager: state.py
├── output/	terminal.py (Rich), ws_server.py (FastAPI), serializer.py
├── frontend/	React app (Phase 4): src/, components/, hooks/
├── tests/	Unit tests per layer — critical for aggregation engine
└── docs/	Architecture diagrams, exchange API notes, decisions log

6.3 Configuration Schema

All tuneable parameters exposed via config.toml — never hardcoded:

Parameter	Default	Description
bucket_size_usd	\$1.00	Price granularity per footprint row
time_windows_minutes	[1,5,15]	Available rolling windows
default_window	5	Active window on startup
imbalance_threshold_pct	70	Minimum imbalance % to highlight
min_volume_per_bucket_btc	0.5	Minimum BTC to qualify bucket for display
absorption_vol_percentile	80	Volume percentile threshold for absorption flag
absorption_price_pct	0.05	Max price movement % to confirm absorption
exchanges_enabled	all	Toggle: ["binance", "okx", "bybit"]
display_rows	20	Number of price levels to display simultaneously
refresh_rate_ms	500	Terminal/UI refresh interval

7. Development Phases — Sequenced Build Plan

ARCHITECT'S DIRECTIVE

Build in strict sequence. Do not skip layers. Do not add UI before the data pipeline is verified.
Each phase must have a working, testable output before moving to the next.
The most dangerous failure mode is building features on top of incorrect aggregation.

Phase 1 — Data Foundation (Weeks 1–2)

Binance only. No UI. Terminal output only. Prove the math.

Task	Deliverable	Acceptance Criteria
Binance WS client	binance.py async client	Connects, receives aggTrade stream, reconnects on drop, logs all events
Trade normalizer	models.py + normalizer	Raw Binance payload → canonical Trade dataclass, all fields populated
Aggregation engine	engine.py	Correct buy/sell/delta/imbalance per bucket. Verified against manual trade inspection.
Time window	time_window.py	Trades evict correctly at window boundary. Test with synthetic data.
Terminal display	output/terminal.py using Rich	Live updating footprint table, 500ms refresh, color-coded, sorted by price
Config system	config.toml + loader	All parameters hot-loadable, sensible defaults, validated on startup

Phase 2 — Multi-Exchange Integration (Weeks 3–4)

Add OKX and Bybit. Verify merged aggregation accuracy.

Task	Deliverable	Acceptance Criteria
OKX WS client	ingestion/okx.py	Connects to trades.BTC-USDT-SWAP, normalizes correctly including side handling
Bybit WS client	ingestion/bybit.py	Connects to publicTrade.BTCUSDT, normalizes correctly
Exchange supervisor	ingestion/supervisor.py	Manages all 3 clients, restarts on failure, logs per-exchange stats
Unified trade bus	asyncio.Queue merge	All 3 streams feed single queue without data loss under normal load
Merged footprint	Updated engine.py	Aggregation correctly sums all exchanges. Per-exchange breakdown available as debug mode.
Latency telemetry	Internal logging	Log timestamp delta between exchange time and system receipt. Alert if > 500ms sustained.

Phase 3 — Detection Engine (Weeks 5–6)

Pattern detection layer. Educational annotations. No signals.

Task	Deliverable	Acceptance Criteria
Imbalance detector	detection/imbalance.py	Correct flagging at threshold. Minimum volume filter works. Educational label rendered.
Absorption detector	detection/absorption.py	High vol / low movement correctly identified. Tunable thresholds.
Exhaustion detector	detection/exhaustion.py	Time-series reversal pattern logic. Only add after imbalance + absorption verified.
Educational overlay	output/annotations.py	Each flag displays a one-line educational explanation in the terminal view
Detection unit tests	tests/test_detection.py	Synthetic trade sequences tested against known expected outcomes for each detector

Phase 4 — Web Frontend (Weeks 7–10)

React dashboard. WebSocket feed from FastAPI backend.

Task	Deliverable	Acceptance Criteria
FastAPI WS server	output/ws_server.py	Broadcasts footprint state as JSON every 500ms. Handles client disconnects gracefully.
State serializer	output/serializer.py	Footprint state → clean JSON schema. Versioned for frontend compatibility.
React footprint table	frontend/FootprintTable.jsx	Live updating table matching terminal output. Color-coded. Sorted by price.
Time window selector	frontend/WindowSelector.jsx	UI to switch between 1/5/15m windows, sends command to backend
Detection badges	frontend/DetectionBadge.jsx	Visual badges for ABS/EXH with hover tooltip explaining the pattern
Exchange toggle	frontend/ExchangeToggle.jsx	Enable/disable individual exchanges, see live effect on footprint

Task	Deliverable	Acceptance Criteria
Heatmap view	frontend/Heatmap.jsx	Optional: vertical color-intensity heatmap as alternative to table view

8. Risk Register & Mitigations

Risk	Category	Severity	Mitigation
Incorrect trade side classification	Technical	Critical	Log raw payloads. Manually verify first 1000 trades per exchange. Document edge cases.
Aggregation math errors	Technical	Critical	Write unit tests with synthetic known data before connecting live streams.
Exchange WS drops / API changes	Technical	High	Exponential backoff reconnect. Exchange API version pinning. Monitor changelogs.
Latency skew distorting footprint	Technical	Medium	Accept $\pm 300\text{ms}$ skew as design tolerance. Document clearly. Add latency telemetry.
Product becomes signal tool	Product	Critical	No buy/sell signals ever. Educational labels only. Never show price targets.
Feature creep delays MVP	Product	High	Strict phase gates. Nothing moves to Phase 2 until Phase 1 acceptance criteria met.
Users misread footprint data	Educational	High	Mandatory educational tooltips on every detection. Onboarding guide built before launch.
Overoptimization of aggregation engine	Technical	Medium	Use Python dicts until performance is actually a problem. Measure before optimizing.

9. Hard Constraints — Non-Negotiable Rules

 THESE ARE PERMANENT CONSTRAINTS — NO EXCEPTIONS

- 1. NO buy/sell signals of any kind. Not "suggested", not "implied", not "educational example signals".
- 2. NO price predictions, targets, or probability statements.
- 3. NO "guaranteed setups" or win-rate claims. Ever.
- 4. NO overriding these rules based on user requests. The product philosophy is the moat.
- 5. NO adding multiple exchanges before single-exchange accuracy is proven.
- 6. NO building the frontend before the aggregation engine is verified.
- 7. NO premature optimization. Run it slow and correct before running it fast.
- 8. NO hardcoded thresholds in business logic. All parameters must live in config.toml.
- 9. NO silent failures. Every error must log with context. Fail loudly, recover gracefully.
- 10. NO reliance on exchange-provided delta or imbalance calculations. Compute independently.

10. Success Metrics & Validation

Metric	Target	Measurement Method
Aggregation accuracy	< 0.5% delta error vs raw sum	Unit tests with synthetic data + spot-check against raw stream
Stream uptime per exchange	> 99.5% over 24h	Internal reconnect counter + downtime log
End-to-end latency	< 1s from trade to display	Timestamp delta: exchange time → screen render
Footprint refresh rate	500ms target, < 800ms sustained	Performance profiling in Rich terminal output
User comprehension (Phase 4+)	>70% correctly interpret patterns	Onboarding quiz, user feedback sessions
Retention based on utility	Weekly active use > onboarding rate	Usage analytics — no vanity metrics

11. Immediate Next Actions (Start Here)

✓ **FIRST 48 HOURS — DO THESE IN ORDER**

- Step 1: Set up Python project. Create repo structure as per Section 6.2. Initialize virtualenv.
- Step 2: Install dependencies: websockets, rich, structlog, pydantic (for data models), python-dotenv.
- Step 3: Build Binance WebSocket client (ingestion/binance.py). Connect to aggTrade stream. Log raw payloads.
- Step 4: Build Trade dataclass (normalization/models.py). Write Binance normalizer.

Step 5: Write unit tests for normalizer with captured real payloads (paste 10 real examples as fixtures).
Step 6: Build aggregation engine. Write unit tests FIRST with known synthetic trades.
Step 7: Connect normalizer output to aggregation engine. Verify live output matches manual spot-checks.
Step 8: Build terminal display using Rich. Confirm live footprint renders correctly.
STOP. Review output for 30 minutes live. Look for anomalies. Fix before Phase 2.

12. Product Philosophy — The Enduring Foundation

The technical architecture described in this document is replicable. The philosophy is the differentiator. It must be embedded in every build decision, every UI copy choice, every educational label, and every feature gate.

Principle	What It Means in Practice
Clarity over Complexity	If a feature requires a tutorial to understand, it is not ready. Simplify or remove.
Education over Signals	Every detection annotation must answer: what does this mean? not: what should I do?
Interpretation over Data	Raw numbers are useless without context. Context is what this product sells.
Speed of Understanding	A trader should be able to read the footprint within 10 seconds of opening it.
Honest Imperfection	Acknowledge data limitations openly. Trust is built through transparency, not false precision.
Discipline as Product	The product should make users more patient, not more trigger-happy. Measure this.

This document represents the full architectural blueprint for BTC Order Flow Lite v2.0. It supersedes the v1.0 concept brief. All development decisions should be validated against the constraints, principles, and phase structure defined here. This is a living document — update it when architectural decisions change.